

National University of Sciences and Technology (NUST)

MS Artificial Intelligence and Autonomous Systems

Sensor-Based Air Pollutant Concentration Regression

Assignment 1: Machine Learning (SE-807)

Student	Muhammad Faheem (583671)
Professor	Dr. Neelma Naz
Date	7th October, 2026

Abstract. This report predicts the hourly CO concentration (CO(GT)) from five metal-oxide sensor signals and three weather variables of the UCI Air Quality dataset. After removing the 2,127 unusable rows, 7,344 hours remain, split 80/20 and standardised. Linear and polynomial regression were written from scratch (NumPy) and verified against scikit-learn; Decision Trees were trained for max_depth 3, 5, 10 and None. All choices were made with week-grouped cross-validation. On the test set the degree-2 polynomial is best (RMSE 0.447 mg/m³, R² 0.901), linear regression follows (RMSE 0.493, R² 0.880) and beats every Decision Tree (best: depth 5, RMSE 0.513). The PT08.S2(NMHC) sensor dominates every importance measure.

Contents

1	Summary of results	4
2	Data preparation	4
2.1	Loading and missing values	4
2.2	Target and feature selection	5
2.3	Class-wise balance of the target	6
2.4	Scaling, split and regularisation decisions	6
3	Model training	7
3.1	Validation protocol	7
3.2	Linear regression from scratch	7
3.3	Polynomial regression and the choice of degree	8
3.4	Decision Tree regressors	8
4	Model evaluation	9
4.1	MSE, RMSE and R ² for all models	9
4.2	Uncertainty: week-clustered bootstrap	9
4.3	Performance metrics versus epoch	10
4.4	Performance metrics versus tree depth	11
4.5	Residual diagnostics	11
4.6	Where the errors occur	13
4.7	Robustness on a chronological split	13
5	Model visualization	13
5.1	Decision tree with max_depth = 3	14
5.2	Feature importance from four methods	14
5.3	Grouped permutation importance	15
5.4	Partial dependence and on-data behaviour	15
6	Baseline comparison: linear regression vs Decision Tree	16
6.1	Head-to-head	16
6.2	Is there non-linear structure?	17
6.3	What each model family can represent (synthetic data)	17
6.4	Where each model wins, and ablations	18
6.5	Scorecard	19
7	Discussion	19
7.1	Limitations and next steps	20
8	Appendix A: code excerpts	20
9	Appendix B: deliverables and reproducibility	22

List of Figures

Figure 1	Missing share per column (left) and missingness over time (right). The 366 blank hours are instrument outages shared by all sensor and weather columns.	5
----------	---	---

Figure 2	Pearson (left) and Spearman (right) correlations of the sensor, weather and target variables.	6
Figure 3	Class-wise balance of the CO regimes and the empirical distribution of CO(GT).	6
Figure 4	Learning-rate sweep for batch gradient descent (left) against the theoretical stability limit (right). ..	7
Figure 5	Degree selection. Left: training RMSE falls monotonically while CV RMSE is U-shaped. Centre and right: crossing the degree with a ridge penalty rescues degrees 4 and 5 but never beats degree 2.	8
Figure 6	Test RMSE and R^2 with 95% week-bootstrap intervals (mean predictor omitted: its RMSE is about three times larger).	10
Figure 7	Train and test MSE, RMSE and R^2 per epoch for batch gradient descent (linear) and Adam (polynomial); dotted lines are the closed-form optimum.	10
Figure 8	Train, CV and test MSE, RMSE and R^2 against tree depth; dotted lines mark the required depths 3, 5, 10 and None (depth 28).	11
Figure 9	Test-set diagnostics for linear, polynomial and depth-5 tree: predicted vs actual, residuals vs fitted, residual distribution and residuals over time.	12
Figure 10	Test RMSE and mean error (prediction – truth) by CO regime for the final models.	13
Figure 11	Mean residual and RMSE by hour of day, and RMSE by month (test set).	13
Figure 12	Decision tree for max_depth = 3 fitted on unscaled features, so thresholds are in sensor units; value is the mean CO(GT) of the node.	14
Figure 13	Importance share by method (left), normalised importance of three models (centre) and linear coefficients with 95% bootstrap intervals (right).	14
Figure 14	Increase in test RMSE when a group of features is permuted jointly.	15
Figure 15	Partial dependence of the four leading features for each model.	15
Figure 16	Binned means of actual and predicted CO along each feature (test set): realistic inputs only.	16
Figure 17	Eight days around the highest test CO value; dots are hours not used for training.	16
Figure 18	Linear, degree-2 and depth-4 tree fits to four synthetic truths (test RMSE in the legends; noise floor 0.15).	17
Figure 19	Extrapolation beyond the training range (shaded) for a linear and a quadratic truth.	17
Figure 20	Feature ablation: CV RMSE with all features, sensors only, weather only, a single sensor, and with each feature dropped.	18

1 Summary of results

Question What was done, and what was found?

Short answer CO(GT) was predicted from eight sensor and weather features. The degree-2 polynomial is the most accurate model; linear regression beats every Decision Tree, but only narrowly for the best-sized tree.

Model	Test RMSE (mg/m ³)	Test R ²	CV RMSE
Mean predictor	1.423	−0.000	1.438
Linear regression	0.493	0.880	0.488
Polynomial, degree 2	0.447	0.901	0.453
Decision Tree, depth 3	0.585	0.831	0.577
Decision Tree, depth 5	0.513	0.870	0.509
Decision Tree, depth 10	0.544	0.854	0.598
Decision Tree, depth None	0.610	0.817	0.659

- **Data.** −200 is the missing-value marker; NMHC (GT) is 90% missing and is excluded; 7,344 hourly rows are used.
- **Most important feature.** PT08.S2 (NMHC), with a mean share of 77% across eight importance measures; the weather variables alone are no better than the mean predictor.
- **Linear vs Decision Tree.** Linear wins: test RMSE 0.493 against 0.513 for the best tree; the relation is a smooth, mostly additive dose-response that a line represents with 9 numbers.
- **Weak spot.** Every model under-predicts the rare very-high CO episodes.

How to read the test scores

The 80/20 split is random and hourly samples are strongly autocorrelated, so test scores are somewhat optimistic. Every choice therefore used week-grouped cross-validation, and the uncertainty is shown with week-clustered bootstrap intervals (Section 4).

2 Data preparation

Question How was the dataset cleaned and prepared for modelling?

Short answer The semicolon-separated file was parsed, −200 was treated as missing, outage rows and rows without the target were dropped (no imputation), features were standardised on the training set only, and the data were split 80/20 stratified on CO level.

2.1 Loading and missing values

The CSV uses ; separators and decimal commas, with 114 blank trailing rows and two empty columns. After removing them, 9,357 hourly records remain (10 Mar 2004 to 4 Apr 2005, no gaps or duplicates). The value −200 appears in every column and is the dataset's missing-value marker; left untreated it would distort every statistic and fit.

Column	Missing	Missing %	Valid
NMHC(GT)	8,443	90.2%	914
CO(GT)	1,683	18.0%	7,674
NO2(GT)	1,642	17.5%	7,715
NOx(GT)	1,639	17.5%	7,718
PT08.S1(CO)	366	3.9%	8,991
C6H6(GT)	366	3.9%	8,991

(continued)

Column	Missing	Missing %	Valid
PT08.S2(NMHC)	366	3.9%	8,991
PT08.S3(NOx)	366	3.9%	8,991
PT08.S4(NO2)	366	3.9%	8,991
PT08.S5(O3)	366	3.9%	8,991
T	366	3.9%	8,991
RH	366	3.9%	8,991
AH	366	3.9%	8,991

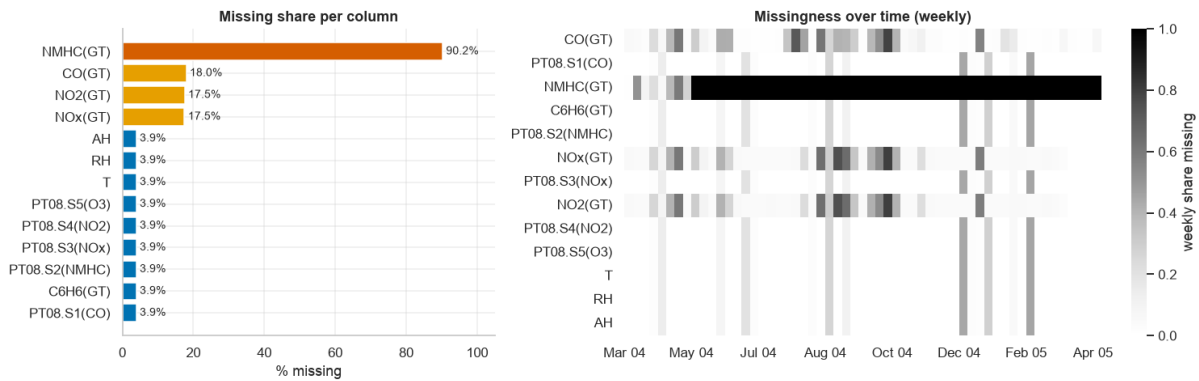


Figure 1. Missing share per column (left) and missingness over time (right). The 366 blank hours are instrument outages shared by all sensor and weather columns.

- NMHC(GT) is 90% missing (only 914 valid hours) and is excluded.
- All sensor and weather columns share the same 366 missing hours; there are no partially missing feature rows, so nothing needs imputing and those rows are dropped.
- CO(GT) is missing for 1,683 hours in total. The target is never imputed: a label filled in by a heuristic would teach the model the heuristic.
- Result: **7,344 usable hourly rows** (78.5% of the cleaned records).

2.2 Target and feature selection

CO(GT) (mg/m^3) is the target, kept in original units. NMHC(GT) is too incomplete; C6H6(GT) is almost a copy of the PT08.S2 sensor ($|r| \approx 0.98$), which would make the task trivial; NOx(GT) and NO2(GT) are about 17% missing and NOx is very skewed. CO(GT) is strongly but not perfectly related to the sensors. The features are the five PT08.* sensors plus T, RH and AH; the other ground-truth pollutants are not used because they would not exist at prediction time.

Feature	Pearson r with CO(GT)	Spearman rho	VIF
PT08.S1(CO)	0.879	0.879	8.1
PT08.S2(NMHC)	0.916	0.927	15.2
PT08.S3(NOx)	-0.703	-0.811	4.1
PT08.S4(NO2)	0.631	0.594	10.3
PT08.S5(O3)	0.854	0.856	8.2
T	0.022	0.069	13.6
RH	0.049	-0.006	7.6
AH	0.049	0.053	10.0

The sensors are strongly collinear (variance inflation factors above 10 for four features), and the weather variables are only weakly related to CO. Spearman exceeds Pearson most clearly for PT08.S3(NOx), a monotonic but non-linear relation.

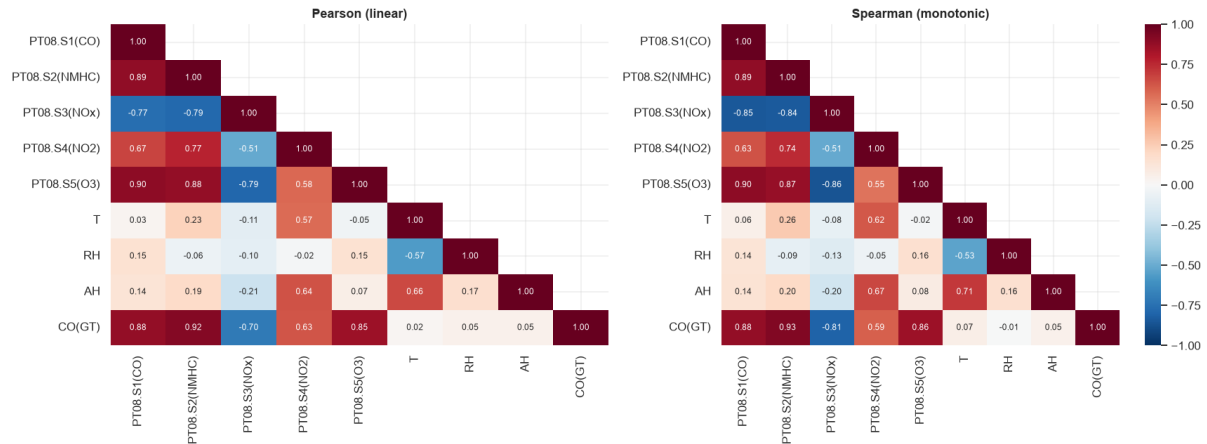


Figure 2. Pearson (left) and Spearman (right) correlations of the sensor, weather and target variables.

2.3 Class-wise balance of the target

CO(GT) is continuous, so its balance is described with four concentration regimes. The regimes are imbalanced (about 8 : 1), which motivates a stratified split and regime-wise error reporting.

CO regime (mg/m ³)	Hours	Share	Train share	Test share
Low (<1.5)	2,820	38.4%	38.4%	38.4%
Moderate (1.5-3)	2,808	38.2%	38.2%	38.3%
High (3-5)	1,369	18.6%	18.6%	18.7%
Very high (>=5)	347	4.7%	4.7%	4.7%

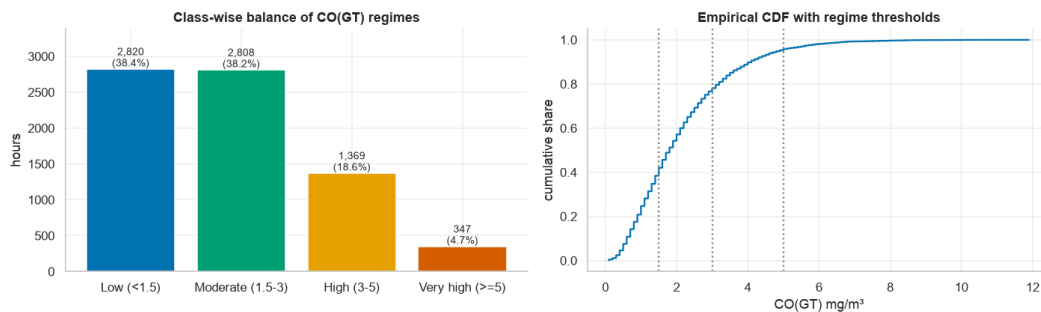


Figure 3. Class-wise balance of the CO regimes and the empirical distribution of CO(GT).

2.4 Scaling, split and regularisation decisions

Aspect	Decision	Why
Missing marker	-200 becomes missing	the minimum is -200 in every column
Outliers	kept	they are real rush-hour pollution episodes, not impossible values
Feature scaling	z-score, fitted on the training set only	scales differ by about 1,000 times; needed for gradient descent and polynomial terms; no test leakage
Target	original units	RMSE stays interpretable; a log target is tested as an ablation
Split	80/20 shuffled, stratified on CO regime, seed 42 (train 5,875, test 1,469)	keeps the test set representative; regime shares match within about 0.1 percentage points
Regularisation	none by default; Ridge studied in Section 3	collinearity; strength is chosen on training data only

Why a shuffled split

A chronological split would train on March 2004 to January 2005 and test on the colder, drier late January to April 2005, a large shift in T , AH and the $PT08.S4$ sensor. The shuffled split is used, with the limitation stated above; a chronological check in Section 4 shows no material loss in RMSE.

3 Model training

Question How were the models trained, and how were their settings chosen?

Short answer Linear and polynomial regression were written in NumPy and checked against scikit-learn; the polynomial degree (2), the learning rate and the tree settings were chosen by week-grouped cross-validation on the training set, never on the test set.

3.1 Validation protocol

Neighbouring hours are near-duplicates, so a plain shuffled K-fold would leak. The training set is split into five folds made of whole calendar weeks (57 weekly groups, `GroupKFold`); no week appears in both a training and a validation fold. All selection (learning rate, degree, ridge strength, tree depth and leaf size, ablations) uses these folds.

3.2 Linear regression from scratch

One NumPy estimator implements ordinary least squares by the normal equation (`lstsq`), batch gradient descent and Adam; the objective is the mean squared error with an optional L2 term that never penalises the intercept. Only NumPy arithmetic is used inside the model: the scikit-learn interface serves purely as a wrapper for the shared cross-validation tools. The code is quoted in Appendix A. The analytic gradient agrees with finite differences to $5e-9$, and both solvers recover known coefficients on synthetic data.

Plain gradient descent is stable only for a learning rate below $2/L$, where L is twice the largest eigenvalue of $X^T X/n$; for the standardised features this limit is 0.238. The sweep agrees: a rate of 0.25 diverges and 0.15 is the fastest stable rate (chosen on training loss only).

Learning rate	Diverged	Final train RMSE	Epochs to 1% of optimum	Distance to OLS coefficients
0.001	no	0.618	990	1.183
0.01	no	0.492	884	0.545
0.05	no	0.480	405	0.024
0.1	no	0.480	203	0.001
0.15	no	0.480	135	0.000
0.25	yes	-	-	-

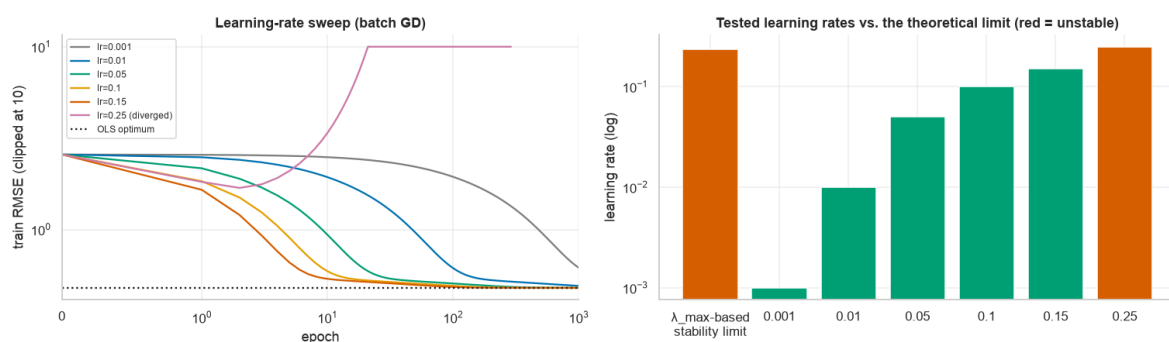


Figure 4. Learning-rate sweep for batch gradient descent (left) against the theoretical stability limit (right).

Linear model	Train RMSE	Test RMSE	Test R ²
Scratch, normal equation	0.4799	0.4933	0.8799
Scratch, gradient descent	0.4799	0.4933	0.8799
scikit-learn LinearRegression	0.4799	0.4933	0.8799

The three implementations give the same coefficients (to 1e-8 for the closed form) and identical metrics. The train-test RMSE gap is small (0.013), so the linear model is limited by bias, not variance.

3.3 Polynomial regression and the choice of degree

A from-scratch feature expansion builds every monomial of total degree 1 to d (it matches `sklearn.preprocessing.Polynomial` exactly); the expanded matrix is re-standardised with training-fold statistics before the least-squares fit. The degree is chosen by cross-validation, using the one-standard-error rule: the smallest degree whose CV RMSE is within one standard error of the best.

Degree	Terms	Train RMSE	CV RMSE	CV RMSE sd	CV R ²
1	8	0.479	0.488	0.061	0.883
2	44	0.407	0.453	0.054	0.899
3	164	0.380	0.500	0.061	0.877
4	494	0.341	1.228	0.809	0.076
5	1286	0.274	8.571	13.349	-93.910

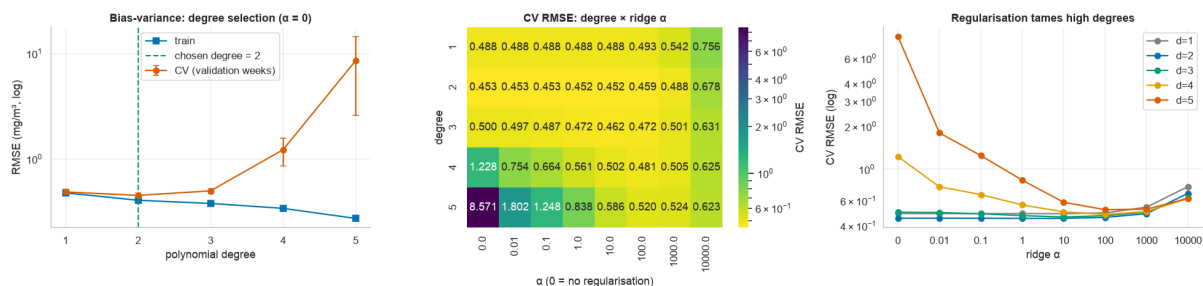


Figure 5. Degree selection. Left: training RMSE falls monotonically while CV RMSE is U-shaped. Centre and right: crossing the degree with a ridge penalty rescues degrees 4 and 5 but never beats degree 2.

Degree 2 is both the CV minimum and the one-standard-error choice. Degrees 4 and 5 (494 and 1,286 collinear terms) extrapolate wildly on unseen weeks. The best combination with ridge is degree 2, alpha 1, with a CV RMSE only marginally lower, so no regularisation is needed. The scratch polynomial, its Adam-trained version (638 epochs to within 1% of the optimum) and the scikit-learn pipeline agree on all metrics (train RMSE 0.4112, test RMSE 0.4470).

3.4 Decision Tree regressors

`DecisionTreeRegressor` was trained for the required depths 3, 5, 10 and None, followed by a depth sweep from 1 to 20, a joint grid over `max_depth` and `min_samples_leaf`, and cost-complexity pruning. The grid with the one-standard-error rule selects depth 5 with `min_samples_leaf` = 50 (29 leaves). Trees are scale-invariant; this was verified, and they were fed the same standardised matrix for a like-for-like comparison.

max_depth	Actual depth	Leaves	Train RMSE	Test RMSE	Test R ²	CV RMSE
3	3	8	0.552	0.585	0.831	0.577
5	5	32	0.440	0.513	0.870	0.509
10	10	693	0.260	0.544	0.854	0.598
None	28	4,245	0.000	0.610	0.817	0.659

Depth 3 under-fits, depth 5 is the best of the four required values, and depth 10 and None over-fit: the

unrestricted tree grows 4,245 leaves, memorises the training set ($R^2 = 1.000$) and loses to depth 5 on the test set by 19%.

4 Model evaluation

Question How well does each model perform, and how certain are the numbers?

Short answer The degree-2 polynomial is best (test RMSE 0.447), followed by linear regression (0.493) and the Decision Trees (0.513 to 0.610); the differences between the top models are small compared with the sampling uncertainty.

4.1 MSE, RMSE and R^2 for all models

Model	Train MSE	Train RMSE	Train R^2	Test MSE	Test RMSE	Test R^2
mean predictor	2.073	1.440	0.000	2.025	1.423	-0.000
Linear scratch (normal eq.)	0.230	0.480	0.889	0.243	0.493	0.880
Linear scratch (GD)	0.230	0.480	0.889	0.243	0.493	0.880
Linear sklearn	0.230	0.480	0.889	0.243	0.493	0.880
Poly(d=2) scratch (normal eq.)	0.169	0.411	0.918	0.200	0.447	0.901
Poly(d=2) scratch (Adam GD)	0.169	0.411	0.918	0.200	0.447	0.901
Poly(d=2) sklearn	0.169	0.411	0.918	0.200	0.447	0.901
Poly(d=2)+Ridge($\alpha=1$) scratch	0.169	0.411	0.918	0.200	0.447	0.901
Poly(d=2)+Ridge($\alpha=1$) sklearn	0.169	0.411	0.918	0.200	0.447	0.901
Tree, max_depth=3	0.304	0.552	0.853	0.343	0.585	0.831
Tree, max_depth=5	0.194	0.440	0.906	0.263	0.513	0.870
Tree, max_depth=10	0.068	0.260	0.967	0.296	0.544	0.854
Tree, max_depth=None	0.000	0.000	1.000	0.372	0.610	0.817
Tree, tuned (CV, 1-SE)	0.221	0.470	0.894	0.266	0.516	0.869
Tree, pruned (ccp, CV)	0.214	0.463	0.897	0.281	0.530	0.861

MSE is in $(\text{mg}/\text{m}^3)^2$, RMSE in mg/m^3 , R^2 is unitless. The scikit-learn and from-scratch versions of the same model are identical to four decimals, so the table holds nine distinct models. Overfitting appears only in the trees: the train-test RMSE gap grows from 0.034 (depth 3) to 0.073 (depth 5), 0.284 (depth 10) and 0.610 (depth None), against 0.013 for linear.

4.2 Uncertainty: week-clustered bootstrap

The test set is resampled by calendar week (57 weeks, 1,000 resamples), because hours inside a week are correlated; all models use the same resamples, so differences between models are paired.

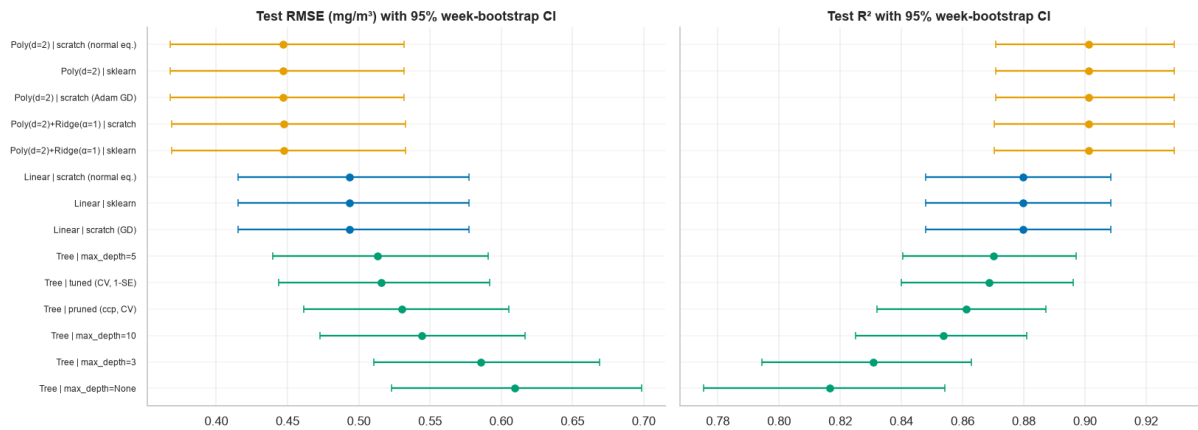


Figure 6. Test RMSE and R^2 with 95% week-bootstrap intervals (mean predictor omitted: its RMSE is about three times larger).

Comparison (A vs B)	Δ RMSE (A – B)	95% interval	P(A better)
Poly d=2 vs Linear	−0.046	[−0.062, −0.032]	100%
Poly d=2 vs Tree tuned	−0.069	[−0.091, −0.044]	100%
Tree depth 5 vs depth 3	−0.072	[−0.101, −0.043]	100%
Tree depth 5 vs depth 10	−0.031	[−0.067, 0.002]	96%
Tree depth 5 vs None	−0.097	[−0.139, −0.056]	100%
Linear vs Poly (d=2)+Ridge	0.046	[0.032, 0.062]	0%

Individual intervals overlap heavily (about ± 0.08 RMSE), so the paired differences are the real comparison. Polynomial beats linear (Δ RMSE -0.046 , interval excludes 0, better in 5 of 5 CV folds). Depth 5 is clearly better than depth 3 and than None but not significantly better than depth 10. Linear against the trees is analysed in Section 6.

4.3 Performance metrics versus epoch

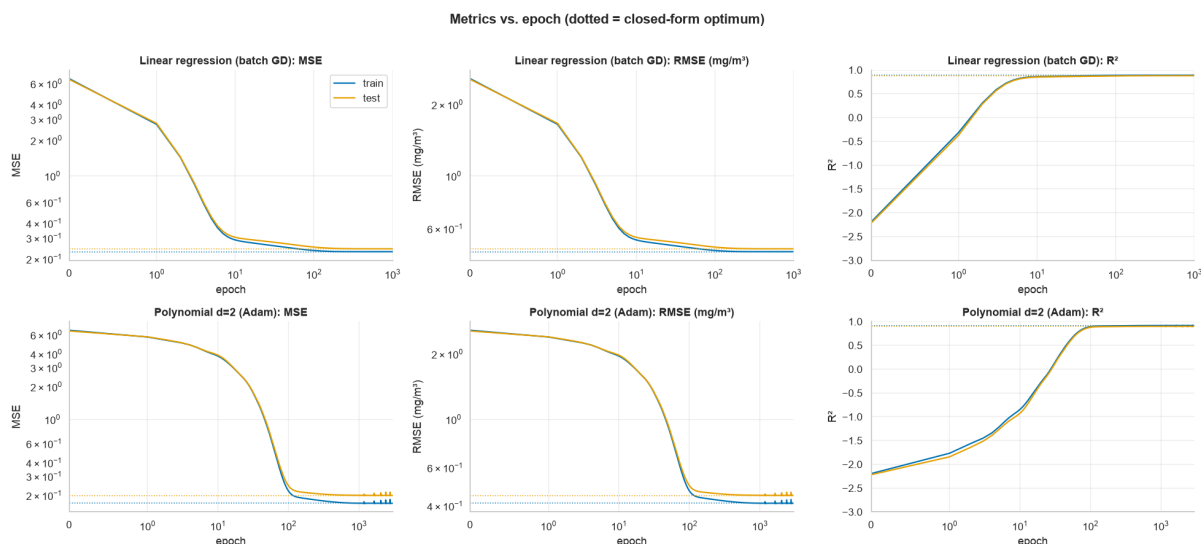


Figure 7. Train and test MSE, RMSE and R^2 per epoch for batch gradient descent (linear) and Adam (polynomial); dotted lines are the closed-form optimum.

Linear gradient descent reaches within 1% of its final training loss after 135 epochs and then follows the closed-form optimum exactly; the polynomial (Adam) needs about 638 epochs. Train and test curves move together at every epoch, so there is no over-training and early stopping would not help. The gradient-based MS Artificial Intelligence and Autonomous Systems, National University of Sciences and Technology (NUST)

fits are about 150 times slower than the closed forms for identical results.

4.4 Performance metrics versus tree depth

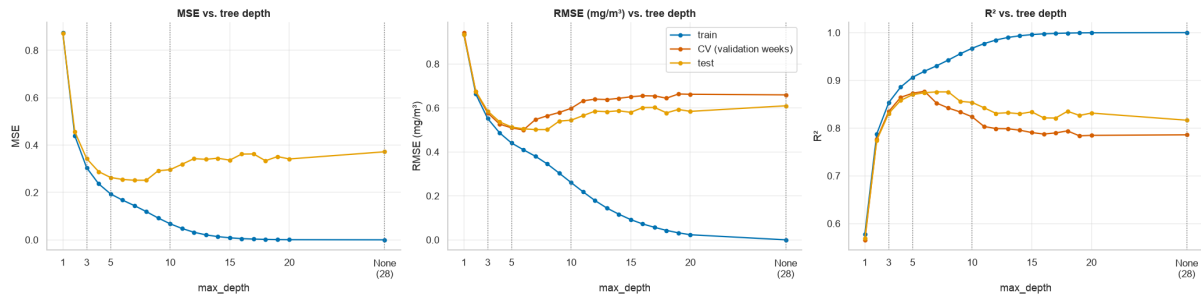


Figure 8. Train, CV and test MSE, RMSE and R^2 against tree depth; dotted lines mark the required depths 3, 5, 10 and None (depth 28).

Training error falls to zero while test and CV error are U-shaped: they drop until depth 5 to 6, then rise as the tree memorises noise. The CV-optimal depth is 6; the test-optimal depth in hindsight is 7, but depths 5 to 8 differ negligibly on the test set.

4.5 Residual diagnostics

Model	Mean residual	sd	Skew	Excess kurtosis	rho(res , fitted)
Linear	0.016	0.493	0.483	5.407	0.241
Poly d=2	0.016	0.447	0.259	7.146	0.383
Tree depth 5	0.016	0.513	0.563	4.665	0.296

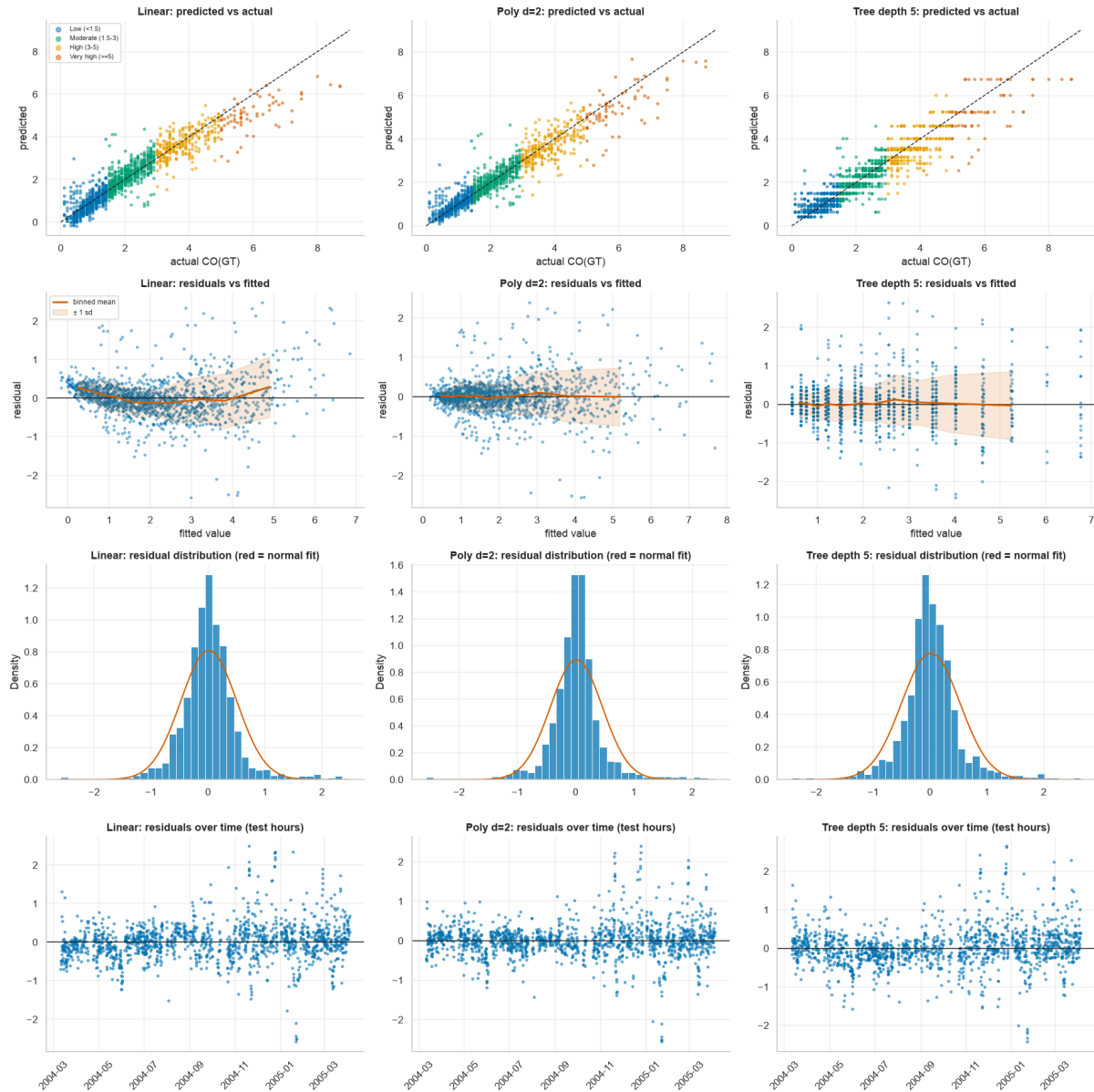


Figure 9. Test-set diagnostics for linear, polynomial and depth-5 tree: predicted vs actual, residuals vs fitted, residual distribution and residuals over time.

- The linear model's residuals show a U-shaped bias against the fitted value (under-prediction at low and high fitted values), direct evidence of non-linearity; the polynomial's are flat.
- The tree predicts a staircase of only 32 distinct values and never exceeds about 6.8 mg/m³.
- All models are heteroscedastic (residual spread roughly doubles from low to high fitted values) and heavy-tailed (excess kurtosis 4.7 to 7.1), so Gaussian error assumptions would be unreliable.

4.6 Where the errors occur

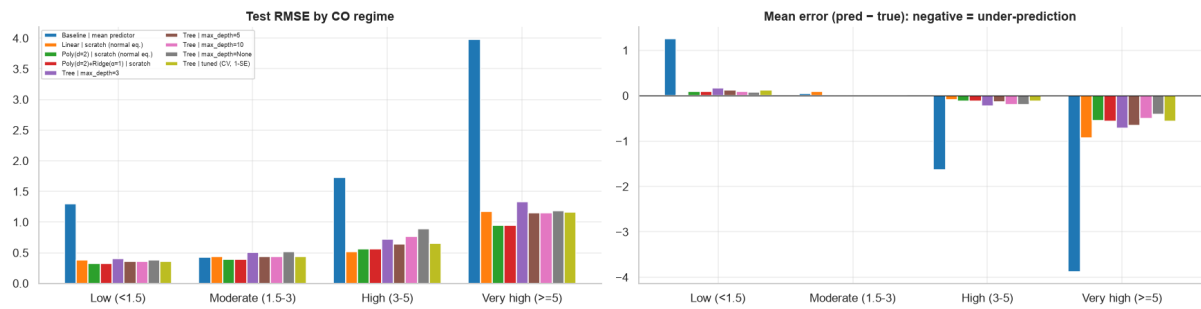


Figure 10. Test RMSE and mean error (prediction – truth) by CO regime for the final models.

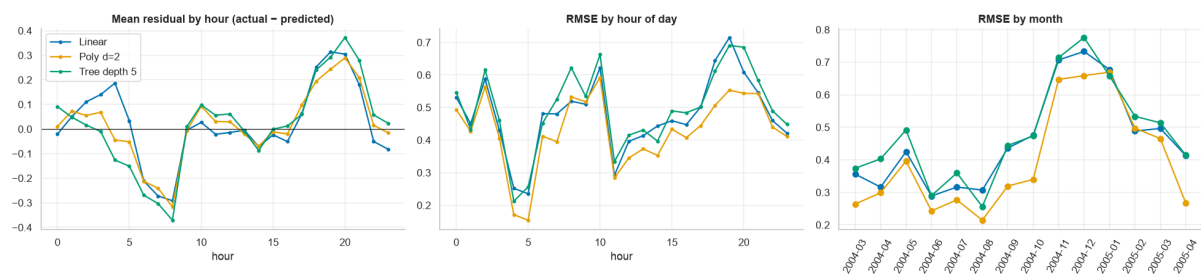


Figure 11. Mean residual and RMSE by hour of day, and RMSE by month (test set).

Every model under-predicts the rare very-high hours (mean error -0.93 for linear, about -0.56 for the polynomial and the tuned tree). All models also under-predict in the evening (18 to 21 h) and over-predict in the early morning (06 to 08 h): hour of day is not a feature, so this is structure the sensors and weather do not carry. RMSE is two to three times larger in November to January than in summer.

4.7 Robustness on a chronological split

Model	Out-of-time test RMSE	Random-split test RMSE	Difference
Mean baseline	1.320	1.423	-0.103
Linear	0.484	0.493	-0.010
Poly($d=2$)	0.446	0.447	-0.001
Poly($d=2$)+Ridge	0.443	0.447	-0.005
Tree max_depth=5	0.528	0.513	0.015
Tree (tuned)	0.513	0.516	-0.003

With hyper-parameters fixed and the first 80% of the timeline used for training, RMSE is essentially unchanged ($|\text{difference}| \leq 0.015$ for the main models) and the ranking is the same. Despite the covariate shift in T, AH and PT08.S4, the CO-sensor relationship is stable across seasons. This is a single split, so it supports rather than proves the conclusion.

5 Model visualization

Question What does the depth-3 tree look like, and which variable contributes most?

Short answer The depth-3 tree splits only on PT08.S2(NMHC); that sensor ranks first in all eight importance measures (mean share 77%), far ahead of temperature and PT08.S1(CO).

5.1 Decision tree with max_depth = 3

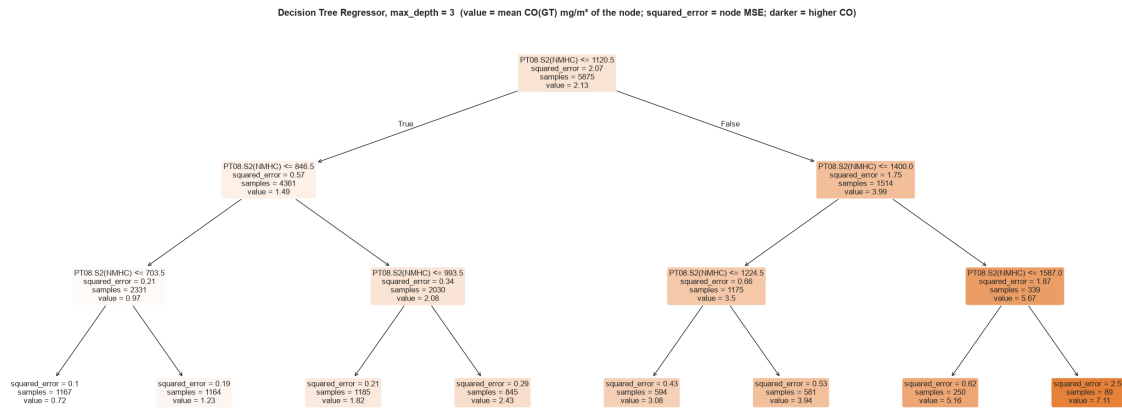


Figure 12. Decision tree for max_depth = 3 fitted on unscaled features, so thresholds are in sensor units; value is the mean CO(GT) of the node.

Leaf rule (PT08.S2 range)	Train hours	Mean CO	sd	Test hours	Test RMSE in leaf
<= 1120 and <= 846 and <= 704	1,167	0.72	0.32	294	0.367
<= 1120 and <= 846 and > 704	1,164	1.23	0.44	290	0.452
<= 1120 and > 846 and <= 994	1,185	1.82	0.46	314	0.483
<= 1120 and > 846 and > 994	845	2.43	0.54	201	0.684
> 1120 and <= 1400 and <= 1224	594	3.08	0.65	144	0.681
> 1120 and <= 1400 and > 1224	581	3.94	0.73	148	0.820
> 1120 and > 1400 and <= 1587	250	5.16	0.79	60	0.895
> 1120 and > 1400 and > 1587	89	7.11	1.61	18	1.132

All seven splits use the same sensor, at thresholds from 704 to 1,587; predicted CO rises monotonically from 0.72 to 7.11 mg/m³. The highest leaf holds only 89 training hours with a large spread, so a single constant represents the extreme episodes badly.

5.2 Feature importance from four methods

Tree impurity, permutation importance on the test set, absolute standardised linear coefficients (with bootstrap intervals) and drop-one retraining are compared, because each has blind spots; sensors are collinear, so redundant sensors share credit.

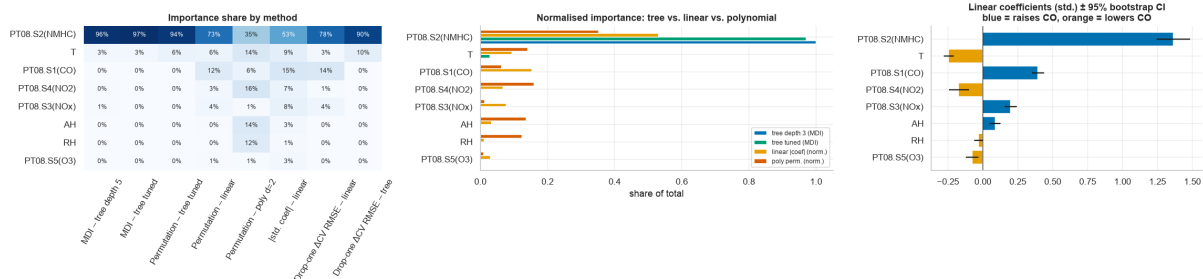


Figure 13. Importance share by method (left), normalised importance of three models (centre) and linear coefficients with 95% bootstrap intervals (right).

Feature	Mean share	Linear coefficient	95% interval
PT08.S2(NMHC)	77.0%	1.366	[1.242, 1.484]
T	6.8%	−0.244	[−0.281, −0.206]

(continued)

Feature	Mean share	Linear coefficient	95% interval
PT08.S1(CO)	6.1%	0.394	[0.354, 0.437]
PT08.S4(NO ₂)	3.4%	-0.174	[-0.241, -0.101]
PT08.S3(NO _x)	2.2%	0.199	[0.156, 0.244]
AH	2.2%	0.087	[0.047, 0.125]
RH	1.7%	-0.030	[-0.061, 0.003]
PT08.S5(O ₃)	0.6%	-0.077	[-0.121, -0.031]

- PT08.S2(NMHC) is rank 1 in all eight methods; temperature (T) and PT08.S1(CO) follow with mean shares of 6.8% and 6.1%.
- PT08.S1(CO) matters for the linear model but not for the trees: it is redundant with PT08.S2 ($r \approx 0.89$), a collinearity effect rather than irrelevance.
- Only RH has a coefficient whose 95% interval includes zero.

5.3 Grouped permutation importance

Correlated columns are shuffled together on the test set, so the model cannot lean on a twin.

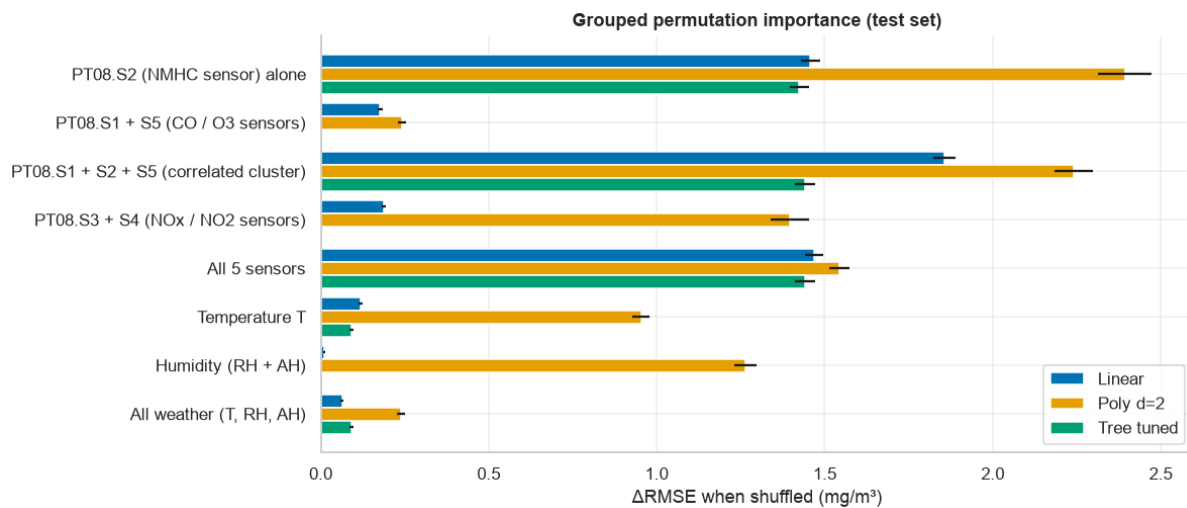


Figure 14. Increase in test RMSE when a group of features is permuted jointly.

Shuffling all five sensors costs the linear model 1.47 mg/m³ of RMSE, against 0.06 for all weather variables. The polynomial model shows large values for non-S2 groups: shuffling creates feature combinations that never occur, and the degree-2 model with many cross-terms reacts violently, a sign of fragility off the data manifold rather than real reliance.

5.4 Partial dependence and on-data behaviour

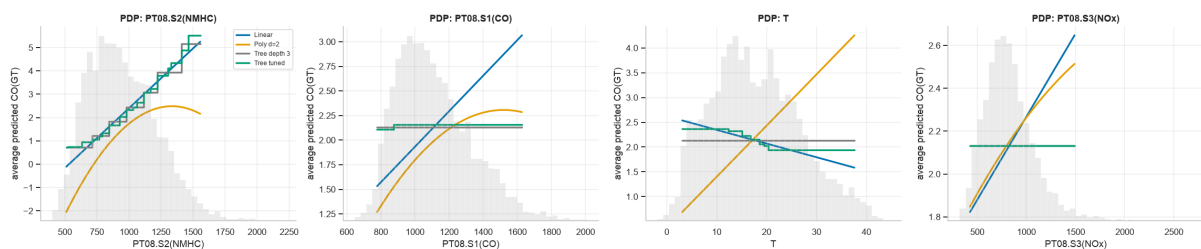


Figure 15. Partial dependence of the four leading features for each model.

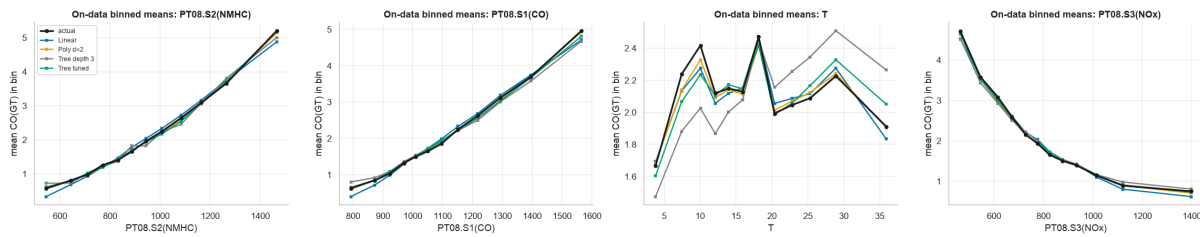


Figure 16. Binned means of actual and predicted CO along each feature (test set): realistic inputs only.

The PDP of the linear model is a line and the trees follow the same line as a staircase for PT08.S2, so its average effect is essentially linear. The polynomial's PDP is implausible (negative CO at low S2) because a PDP evaluates the model at unrealistic combinations. On the actual data the polynomial is the most faithful (largest binned deviation 0.09 mg/m³ against 0.13 tuned tree, 0.23 linear, 0.26 depth-3 tree); the linear model under-estimates at both ends of the sensor ranges.

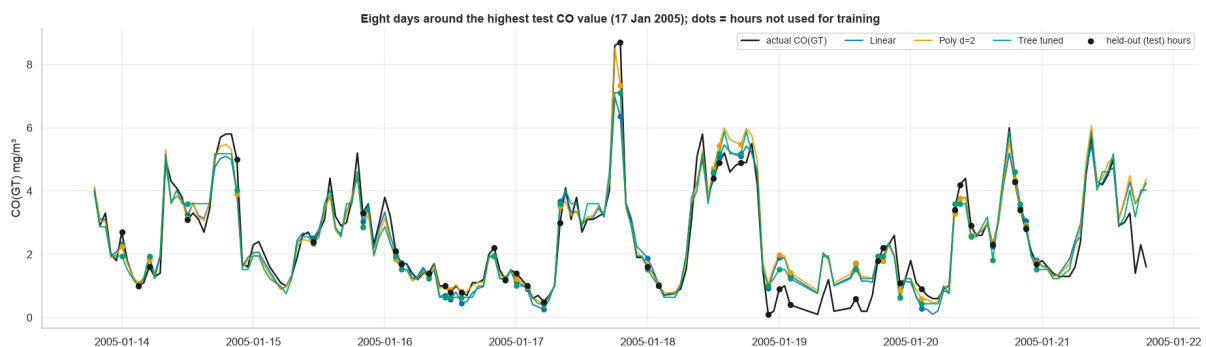


Figure 17. Eight days around the highest test CO value; dots are hours not used for training.

6 Baseline comparison: linear regression vs Decision Tree

Question Does a Decision Tree capture structure that linear regression misses?

Short answer No: linear regression is more accurate than every tree configuration (test RMSE 0.493 against 0.513 for the best tree). The non-linearity in the data is real but smooth and mild, and is captured best by a degree-2 polynomial.

6.1 Head-to-head

Decision Tree	Leaves	Test RMSE	Test R ²	Δ RMSE vs linear	95% interval	Linear wins (folds of 5)
max_depth=3	8	0.585	0.831	0.092	[0.053, 0.128]	5
max_depth=5	32	0.513	0.870	0.020	[-0.001, 0.039]	5
max_depth=10	693	0.544	0.854	0.051	[0.017, 0.087]	5
max_depth=None	4,245	0.610	0.817	0.116	[0.078, 0.154]	5
tuned (CV, 1-SE)	29	0.516	0.869	0.022	[0.000, 0.044]	4
pruned (ccp, CV)	21	0.530	0.861	0.037	[0.014, 0.060]	-

Linear regression (9 coefficients) has test RMSE 0.493 and R² 0.880. The gap to the best tree (depth 5) is 0.020 RMSE, with an interval that just touches zero; every other tree is clearly worse. Linear wins in 5 of 5 CV folds against every fixed-depth tree and in 4 of 5 against the tuned tree. The trees also generalise worse (train-test gap up to 0.610 against 0.013).

6.2 Is there non-linear structure?

A nested F-test strongly prefers the degree-2 model over the linear one ($F = 58.7$; the p-value underflows to zero and is optimistic under autocorrelation). The held-out evidence is more convincing: test R^2 rises from 0.880 to 0.901 and CV RMSE falls from 0.488 to 0.453.

Feature set	Linear	Poly d=2	Tuned tree
Best single sensor (PT08.S2)	0.575	0.520	0.534
All 8 features	0.488	0.453	0.513
Relative gain from adding the other seven features	15.1%	12.8%	3.9%

CV RMSE with PT08.S2 alone and with all features. A tree is good at the curvature of a single variable (alone, the tuned tree at 0.534 beats the line at 0.575) but inefficient at adding several variables up: each split uses one feature, so adding the other seven features improves the linear model by 15% and the tree by only 4%.

As a probe, a shallow tree was fitted to the residuals of the linear model (an extension, not an assignment model): test RMSE falls from 0.493 to 0.470, confirming that the line leaves non-linear signal, yet it does not reach the smooth polynomial (0.447).

6.3 What each model family can represent (synthetic data)

One input, noise sd 0.15 and four known truths; each model is fitted to 70% of the points and tested on the rest, and then trained on x in $[0, 1]$ and tested on $[1, 1.5]$. This isolates the mechanisms behind the real-data result.

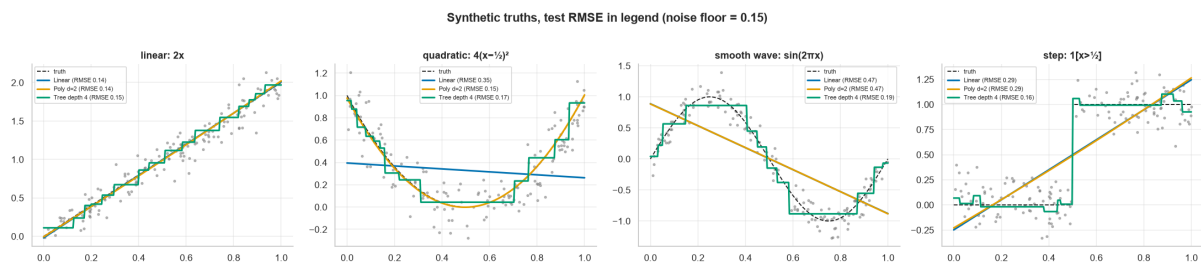


Figure 18. Linear, degree-2 and depth-4 tree fits to four synthetic truths (test RMSE in the legends; noise floor 0.15).

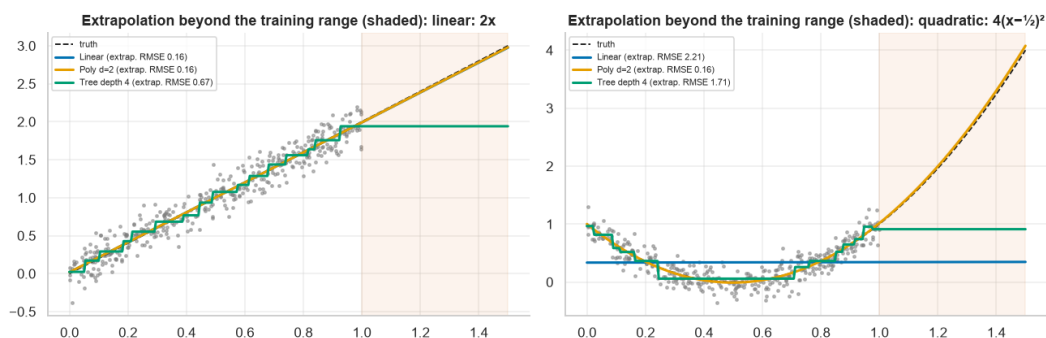


Figure 19. Extrapolation beyond the training range (shaded) for a linear and a quadratic truth.

Truth	Linear	Poly d=2	Tree depth 4	Linear (extrap.)	Poly (extrap.)	Tree (extrap.)
linear: $2x$	0.14	0.14	0.15	0.16	0.16	0.67
quadratic: $4(x-\frac{1}{2})^2$	0.35	0.15	0.17	2.21	0.16	1.71
smooth wave: $\sin(2\pi x)$	0.47	0.47	0.19	-	-	-
step: $1[x > \frac{1}{2}]$	0.29	0.29	0.16	-	-	-

Trees win on abrupt or irregular shapes (sine, step); the degree-2 polynomial wins on the quadratic; a straight line fails on every non-linear truth; and outside the training range a tree freezes at its last leaf value. The air-quality relation resembles the smooth, nearly linear or quadratic case, which is why linear and polynomial models win.

6.4 Where each model wins, and ablations

PT08.S2 decile	Mean CO	Linear	Poly d=2	Tree depth 5	Tuned tree	Best
D1	0.61	0.300	0.149	0.218	0.218	Poly
D2	0.88	0.411	0.361	0.428	0.428	Poly
D3	1.07	0.399	0.371	0.416	0.416	Poly
D4	1.36	0.385	0.376	0.401	0.401	Poly
D5	1.64	0.382	0.361	0.388	0.388	Poly
D6	1.97	0.350	0.320	0.391	0.391	Poly
D7	2.36	0.525	0.508	0.548	0.548	Poly
D8	2.88	0.539	0.507	0.589	0.589	Poly
D9	3.48	0.628	0.609	0.642	0.642	Poly
D10	4.98	0.800	0.672	0.842	0.857	Poly

Test RMSE per decile of the dominant sensor. The polynomial is best in all ten deciles; between the two models asked about, linear beats the tuned tree in nine of ten (the tree wins only in the lowest decile, where the relation flattens).

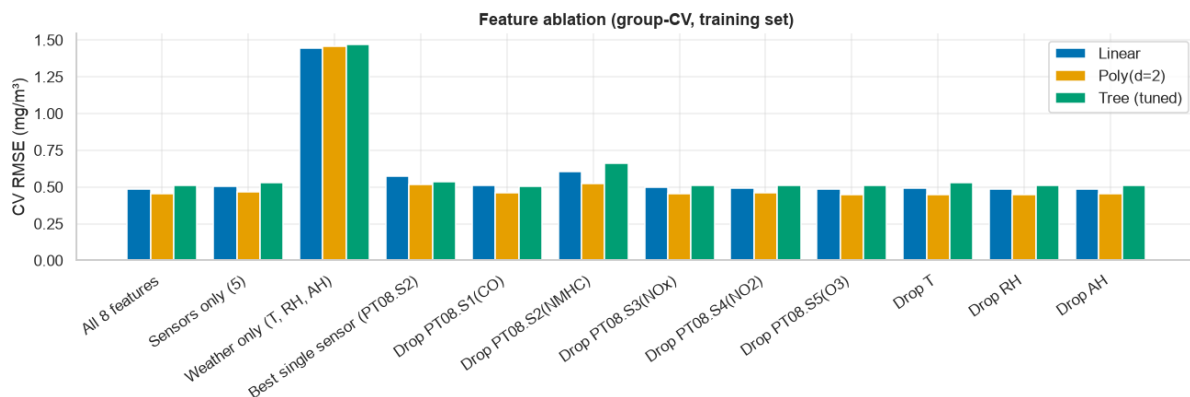


Figure 20. Feature ablation: CV RMSE with all features, sensors only, weather only, a single sensor, and with each feature dropped.

Ablation	Change	Model	Δ	Verdict
Features	sensors only (drop weather)	Linear	+0.014	negligible (within noise)
Features	weather only (drop sensors)	Linear	+0.954	worse
Features	drop PT08.S2 only	Linear	+0.119	worse
Features	PT08.S2 as the only feature	Linear	+0.087	worse
Features	sensors only (drop weather)	Poly(d=2)	+0.016	negligible (within noise)
Features	weather only (drop sensors)	Poly(d=2)	+1.003	worse
Features	drop PT08.S2 only	Poly(d=2)	+0.071	worse
Features	PT08.S2 as the only feature	Poly(d=2)	+0.067	worse
Features	sensors only (drop weather)	Tree (tuned)	+0.014	negligible (within noise)
Features	weather only (drop sensors)	Tree (tuned)	+0.957	worse
Features	drop PT08.S2 only	Tree (tuned)	+0.151	worse
Features	PT08.S2 as the only feature	Tree (tuned)	+0.021	negligible (within noise)
Scaling	no standardisation (batch GD, 1000 epochs)	Linear	+0.141	worse
Target	log1p-transformed target	Linear	+0.036	worse
Target	log1p-transformed target	Poly(d=2)	-0.004	negligible (within noise)
Target	log1p-transformed target	Tree (tuned)	-0.001	negligible (within noise)
Tree criterion	absolute_error instead of squared_error	Tree (tuned)	-0.012	negligible (within noise)
Tree criterion	poisson instead of squared_error	Tree (tuned)	-0.007	negligible (within noise)
Regularisation	ridge (best $\alpha \approx 22$) instead of OLS	Linear	-0.000	negligible (within noise)
Regularisation	ridge $\alpha=1$ on degree 2	Poly(d=2)	-0.001	negligible (within noise)
Polynomial degree	degree 1 instead of 2	Poly	+0.035	worse

(continued)

Ablation	Change	Model	Δ	Verdict
Polynomial degree	degree 3 instead of 2	Poly	+0.047	worse
Polynomial degree	degree 4 instead of 2	Poly	+0.775	worse
Polynomial degree	degree 5 instead of 2	Poly	+8.118	worse
Tree depth	max_depth=3 instead of 5	Tree	+0.068	worse
Tree depth	max_depth=10 instead of 5	Tree	+0.089	worse
Tree depth	max_depth=None instead of 5	Tree	+0.150	worse
Tree regularisation	cost-complexity pruning vs. tuned depth/leaf	Tree	+0.008	negligible (within noise)
Tree regularisation	min_samples_leaf=50 added to depth 5	Tree	+0.004	negligible (within noise)
Hybrid extension	linear + depth-5 tree on residuals	Linear	-0.017	negligible (within noise)
Evaluation protocol	chronological instead of shuffled split	Linear	-0.010	negligible (within noise)
Evaluation protocol	chronological instead of shuffled split	Poly(d=2)	-0.001	negligible (within noise)
Evaluation protocol	chronological instead of shuffled split	Tree max_depth=5	+0.015	negligible (within noise)
Evaluation protocol	chronological instead of shuffled split	Tree (tuned)	-0.003	negligible (within noise)

Positive Δ means the ablated variant is worse than the reference (CV RMSE, training RMSE for the scaling row, test RMSE for the evaluation protocol); differences below 0.03 are within fold-to-fold noise. Sensors alone are as good as all features; weather alone is as bad as the mean predictor; degree 2 and tree depth 5 are the right sizes; ridge, tree criterion, pruning and the chronological split change little; skipping standardisation hurts gradient descent; a log target hurts the linear model.

6.5 Scorecard

Measure	Linear	Tree depth 5	Tuned tree	Poly d=2
test RMSE	0.493	0.513	0.516	0.447
test R ²	0.880	0.870	0.869	0.901
CV RMSE	0.488	0.509	0.513	0.453
CV fold-to-fold sd	0.061	0.057	0.053	0.054
out-of-time test RMSE	0.4836	0.5276	0.5126	0.4460
RMSE gap (test – train)	0.013	0.073	0.046	0.036
RMSE, High (3-5)	0.519	0.646	0.653	0.559
RMSE, Very high (≥ 5)	1.173	1.152	1.161	0.946
bias, Very high (≥ 5)	-0.927	-0.660	-0.556	-0.555
parameters / leaves	9.000	32.000	29.000	45.000
fit time (s)	0.0012	0.0098	0.0102	0.0045

7 Discussion

Question Which features were most important for predicting CO(GT)?

Short answer PT08.S2(NMHC) by far: rank 1 in all eight importance measures (mean share 77%), the only feature the depth-3 tree uses, and removing it raises the CV RMSE by 0.119 (linear), 0.071 (polynomial) and 0.151 (tree). Temperature and PT08.S1(CO) follow.

- **Linear view.** The standardised coefficient of PT08.S2(NMHC) is 1.366 (95% interval 1.242 to 1.484), about 3.5 times the next largest effect.
- **Sensors versus weather.** Using only the five sensors costs just 0.014 RMSE; weather alone is no better than predicting the mean (CV RMSE 1.44 against 1.44). Temperature is the only weather variable with a measurable effect.
- **Collinearity.** PT08.S1(CO) and PT08.S5(O3) correlate strongly with PT08.S2; importance of redundant sensors is shared arbitrarily, so the finding should be read as the sensor cluster led by PT08.S2.

Question Which performed better overall: Decision Tree or Linear Regression? Why?

Short answer Linear regression, although the margin over the best-sized tree is small (test RMSE 0.493 against 0.513). The degree-2 polynomial (0.447) was the best model of all.

	Linear	Best tree (depth 5)	Tuned tree	Poly d=2
Test RMSE (mg/m ³)	0.493	0.513	0.516	0.447
Test R ²	0.880	0.870	0.869	0.901
CV RMSE	0.488	0.509	0.513	0.453
Train-test RMSE gap	0.013	0.073	0.046	0.036
Parameters / leaves	9	32	29	45

- **Strength of the evidence.** The best tree is 0.020 RMSE worse than linear (95% interval -0.001 to 0.039 ; linear better in 97% of resamples) and linear wins 5 of 5 CV folds; against other tree depths the gap is larger and its interval excludes zero.
- **A smooth, additive signal.** CO rises smoothly and almost linearly with PT08.S2, with additive contributions from other sensors and temperature. A line represents this with 9 numbers; a tree approximates it with a staircase of constant steps (the depth-3 tree is an 8-step function of one sensor), losing precision inside every step.
- **Trees add variables up inefficiently.** With PT08.S2 alone the tuned tree beats the line, but adding the other seven features improves linear regression by 15% and the tree by only 4%.
- **Bias and variance.** Shallow trees under-fit, deep trees fit the noise (depth None: training $R^2 = 1.00$, test R^2 0.817); linear regression has low variance and, for this near-linear signal, little bias.
- **Where trees do win.** In the lowest sensor decile and on abrupt shapes (synthetic study); they need no scaling and give readable rules.
- **Linear versus non-linear patterns.** The linear model's residuals show a slight U-shape, a quadratic term gives a real gain, and a tree fitted to the linear residuals also helps: the non-linearity is real but smooth and mild, captured best by the degree-2 polynomial, second best by a linear backbone with tree corrections, and least efficiently by a stand-alone staircase.

7.1 Limitations and next steps

- The random 80/20 split on autocorrelated hourly data makes test scores somewhat optimistic; this was mitigated by week-grouped CV, week-clustered bootstrap intervals and a chronological check (a single split).
- One site, about 13 months and one sensor array; the PT08.S4 sensor drifts over time, so a deployed model would need recalibration.
- Differences among the top models are small relative to uncertainty; coefficients and single-feature importances of correlated sensors are unreliable; the polynomial model is fragile on unrealistic feature combinations.
- Next steps: hour-of-day and day-of-week features, ensembles (random forest, gradient boosting), quantile or heteroscedastic regression for extreme episodes, time-aware nested CV, sensor-drift monitoring.

8 Appendix A: code excerpts

Question Where is the code?

Short answer The complete code and outputs are in the Jupyter notebook `src/notebooks/air_quality_regression.ipynb` (and its PDF export `docs/report/air_quality_regression_notebook.pdf`); the central from-scratch parts are quoted here, copied from the notebook cells.

A.1 Evaluation metrics from scratch

```
# ---- evaluation metrics implemented from scratch (verified against sklearn below) ----
def mse(y_true, y_pred): return float(np.mean((np.asarray(y_true, float) - np.asarray(y_pred, float)) ** 2))
def rmse(y_true, y_pred): return float(np.sqrt(mse(y_true, y_pred)))
def mae(y_true, y_pred): return float(np.mean(np.abs(np.asarray(y_true, float) - np.asarray(y_pred, float))))
def r2(y_true, y_pred):
    y_true, y_pred = np.asarray(y_true, float), np.asarray(y_pred, float)
    return float(1 - np.sum((y_true - y_pred) ** 2) / np.sum((y_true - y_true.mean()) ** 2))
```

```
def regression_metrics(y_true, y_pred):
    return {"mse": mse(y_true, y_pred), "rmse": rmse(y_true, y_pred), "r2": r2(y_true, y_pred), "mae": mae(y_true, y_pred)}
```

A.2 Linear regression from scratch (normal equation, gradient descent, Adam)

```
class LinearRegressionScratch(RegressorMixin, BaseEstimator):
    def __init__(self, solver="normal", alpha=0.0, lr=0.1, epochs=1000, beta1=0.9, beta2=0.999, track=False):
        self.solver, self.alpha, self.lr, self.epochs = solver, alpha, lr, epochs
        self.beta1, self.beta2, self.track = beta1, beta2, track

    @staticmethod
    def _design(X): return np.c_[np.ones(len(X)), np.asarray(X, float)]

    def _grad(self, Xb, y, theta):
        g = (2.0 / len(y)) * Xb.T @ (Xb @ theta - y)
        g[1:] += (2.0 * self.alpha / len(y)) * theta[1:] # L2 term, intercept excluded
        return g

    def fit(self, X, y, eval_sets=None):
        Xb, y = self._design(X), np.asarray(y, float).ravel(); d = Xb.shape[1]
        if self.solver == "normal":
            if self.alpha == 0:
                theta = np.linalg.lstsq(Xb, y, rcond=None)[0]
            else:
                P = self.alpha * np.eye(d); P[0, 0] = 0
                theta = np.linalg.solve(Xb.T @ Xb + P, Xb.T @ y)
        elif self.solver in ("gd", "adam"):
            theta = self._descend(Xb, y, eval_sets)
        else:
            raise ValueError(self.solver)
        self.theta_, self.intercept_, self.coef_ = theta, theta[0], theta[1:]
        return self

    def _descend(self, Xb, y, eval_sets):
        theta, m, v = np.zeros(Xb.shape[1]), np.zeros(Xb.shape[1]), np.zeros(Xb.shape[1])
        ev = {k: (self._design(Xe), np.asarray(ye, float).ravel()) for k, (Xe, ye) in (eval_sets or {}).items()}
        rows, self.diverged_ = [], False

        def record(ep):
            r = {"epoch": ep}
            for nm, (A, t) in {"train": (Xb, y), **ev}.items():
                p = A @ theta
                r[f"{nm}_mse"], r[f"{nm}_rmse"], r[f"{nm}_r2"] = mse(t, p), rmse(t, p), r2(t, p)
            rows.append(r)

        with np.errstate(all="ignore"):
            if self.track: record(0)
            for t in range(1, self.epochs + 1):
                g = self._grad(Xb, y, theta)
                if self.solver == "gd":
                    theta = theta - self.lr * g
                else: # Adam (bias-corrected)
                    m = self.beta1 * m + (1 - self.beta1) * g
                    v = self.beta2 * v + (1 - self.beta2) * g ** 2
                    theta = theta - self.lr * (m / (1 - self.beta1 ** t)) / (np.sqrt(v / (1 - self.beta2 ** t)) + 1e-8)
                if not np.all(np.isfinite(theta)) or np.abs(theta).max() > 1e12:
                    self.diverged_ = True; break
                if self.track: record(t)
            self.history_ = pd.DataFrame(rows) if self.track else None
            return theta

    def predict(self, X): return self._design(X) @ self.theta_
```

A.3 Polynomial features and polynomial regression

```
class PolynomialFeaturesScratch:
    """All monomials of total degree 1..d (no bias column); column order identical to sklearn's."""
    def __init__(self, degree): self.degree = degree
    def fit(self, X, y=None):
        self.n_features_ = np.asarray(X).shape[1]
        self.combos_ = [c for k in range(1, self.degree + 1) for c in combinations_with_replacement(range(self.n_features_), k)]
        return self
    def transform(self, X):
        X = np.asarray(X, float)
        return np.column_stack([np.prod(X[:, c], axis=1) for c in self.combos_])
    def feature_names(self, names):
        return ["*".join(names[i] for i in c) for c in self.combos_]
```

```

class PolyRegressionScratch(RegressorMixin, BaseEstimator):
    def __init__(self, degree=2, solver="normal", alpha=0.0, lr=0.05, epochs=2000, track=False):
        self.degree, self.solver, self.alpha, self.lr, self.epochs, self.track = degree, solver, alpha, lr, epochs, track
    def _expand(self, X): return (self.poly_.transform(X) - self.mu_) / self.sd_
    def fit(self, X, y, eval_sets=None):
        X = np.asarray(X, float); self.poly_ = PolynomialFeaturesScratch(self.degree).fit(X)
        P = self.poly_.transform(X); self.mu_, self.sd_ = P.mean(0), P.std(0); self.sd_[self.sd_ == 0] = 1.0
        ev = {k: (self._expand(Xe), ye) for k, (Xe, ye) in (eval_sets or {}).items()}
        self.lin_ = LinearRegressionScratch(self.solver, self.alpha, self.lr, self.epochs, track=self.track).fit((P - self.mu_) /
self.sd_, y, ev)
        self.intercept_, self.coef_, self.history_ = self.lin_.intercept_, self.lin_.coef_, getattr(self.lin_, "history_", None)
        return self
    def predict(self, X): return self.lin_.predict(self._expand(np.asarray(X, float)))

```

A.4 Degree selection by week-grouped cross-validation

```

def poly_cv_curve(degrees, alphas):
    """CV curve for polynomial (ridge) regression. The expansion is deterministic, so it is done once per degree;
    standardisation, fit and scoring are done per fold with training-fold statistics only (no leakage)."""
    rows = []
    for deg in degrees:
        P = PolynomialFeaturesScratch(deg).fit(Xv).transform(Xv)
        for k, (tr, va) in enumerate(folds):
            mu, sd = P[tr].mean(0), P[tr].std(0); sd[sd == 0] = 1
            A, B = (P[tr] - mu) / sd, (P[va] - mu) / sd
            ym = yv[tr].mean(); G, c = A.T @ A, A.T @ (yv[tr] - ym) # centred columns -> intercept = mean(y)
            for a in alphas:
                w = np.linalg.lstsq(A, yv[tr] - ym, rcond=None)[0] if a == 0 else np.linalg.solve(G + a * np.eye(G.shape[0]), c)
                rows.append({"degree": deg, "alpha": a, "fold": k, "n_terms": P.shape[1],
                    "train_rmse": rmse(yv[tr], A @ w + ym), "val_rmse": rmse(yv[va], B @ w + ym), "val_r2": r2(yv[va], B @
w + ym)})
    return pd.DataFrame(rows)

```

A.5 Decision Tree depth experiment

```

REQ_DEPTHS = [3, 5, 10, None]
tree_names = {}
for d in REQ_DEPTHS:
    nm = f"Tree | max_depth={d}"; tree_names[d] = nm
    m, t = timed_fit(DecisionTreeRegressor(max_depth=d, random_state=SEED), X_train_s, y_train)
    evaluate(nm, m, "Decision Tree", f"max_depth={d}", t, complexity=m.get_n_leaves())
    RESULTS[nm].update({"depth_actual": m.get_depth(), "n_leaves": m.get_n_leaves(),
        **dict(cv_eval(DecisionTreeRegressor(max_depth=d, random_state=SEED)))})

```

A.6 Week-clustered bootstrap

```

test_weeks = pd.factorize(X_test.index.to_period("W").astype(str))[0]
members = [np.where(test_weeks == w)[0] for w in np.unique(test_weeks)]
rb = np.random.default_rng(SEED)
BOOT_IDX = [np.concatenate([members[i] for i in rb.integers(0, len(members), len(members))]) for _ in range(1000)]
yt_all = y_test.values

def boot_dist(name):
    e = yt_all - PRED[name]["test"]; rm, rr = [], []
    for idx in BOOT_IDX:
        s_res = np.sum(e[idx] ** 2); rm.append(np.sqrt(s_res / len(idx))); rr.append(1 - s_res / np.sum((yt_all[idx] -
yt_all[idx].mean()) ** 2))
    return np.array(rm), np.array(rr)

```

9 Appendix B: deliverables and reproducibility

Requirement of the brief	Where
Data preprocessing and handling of missing values	Section 2
Load the data; choose target; select features; split 80/20	Section 2
Linear and polynomial regression from scratch, degree choice, scikit-learn cross-check	Section 3, Appendix A

(continued)

Requirement of the brief	Where
Decision Tree training with max_depth 3, 5, 10, None	Section 3
MSE, RMSE and R ² for all models	Section 4
Performance metrics versus epoch; versus tree depth	Section 4
Decision tree plot for max_depth = 3; feature importances	Section 5
Linear regression compared with the Decision Tree; linear vs non-linear patterns	Section 6
Short discussion: important features; Decision Tree vs Linear Regression	Section 7
Neat, commented code	Appendix A and the notebook

The notebook runs top to bottom from the raw CSV in about 30 seconds (Python 3.11.9, numpy 2.4.6, pandas 3.0.6, scikit-learn 1.9.1, random seed 42). This report is generated from the executed notebook by `src/report/`.